Visualising Sample Data Distributions

Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

PROJECT SOURCE CODE

Submitted by:
Shreyansh Tiwari
Roll No: 25SCS1003001300

Under the supervision of:
Mr. Ramavath Ganesh

Department of Computer Science and Engineering
IILM University

November 2025

Complete Source Code Compilation

Project: Student Data Analytics & Machine Learning Pipeline
Files Included: 3 source files (2 Python scripts, 1 CSV dataset)
Date Generated: November 27, 2025

Table of Contents

1. app.py - Flask Web Application
2. generate_plots_and_models.py - Data Processing & ML Models
3. student_dataset_100.csv - Student Dataset (Sample)

File 1: app.py

Purpose: Flask web application that serves an interactive dashboard displaying student data and generated visualizations.

Description: This is a lightweight Flask server that loads the student dataset, renders it as a table, and serves pre-generated plot images to a web interface. It establishes two routes: one for the homepage displaying top 10 students and plot thumbnails, and another for serving individual plot images.

Complete Source Code (Line-by-Line)

Line 1-2: Import Statements

from flask import Flask, render_template, send_from_directory
import pandas as pd, os
Explanation:

- Flask: Core framework for creating the web application
- render_template: Renders HTML templates from the templates/ directory
- send_from_directory: Safely serves static files (PNG images) from disk
- pandas (pd): Data manipulation and CSV reading
- os: Operating system interface for file paths and directory operations

Line 4: Flask App Initialization

app=Flask(**name**)
Explanation: Creates a Flask application instance. __name__ provides the module name for Flask to locate resources. The variable app is the main application object that handles all routing and requests.

Line 6: Base Directory Path

base=os.path.dirname(**file**)
Explanation: Stores the absolute path to the current script's directory. This allows relative paths to work correctly regardless of where the script is executed from. Essential for portable file handling.

Line 8: Load CSV Dataset

df=pd.read_csv(os.path.join(base,"student_dataset_100.csv"))
Explanation:

- pd.read_csv(): Reads CSV file into a pandas DataFrame
- os.path.join(): Constructs OS-agnostic file path
- df: DataFrame containing 100 student records with 13 columns (student_id, name, age, gender, grade, IQ, total_fees, attendance, cgpa, city, fees_paid, skills, fees_outstanding)

Lines 10-11: Homepage Route Definition

@app.route("/")
def home():
Explanation:

- @app.route("/"): Decorator that maps HTTP GET requests to root URL / to the home() function
- def home():: Function definition that handles requests to the homepage

Line 12: Extract Top 10 Records for Display

table=df.head(10).to_dict(orient="records")
Explanation:

- df.head(10): Selects first 10 rows from the dataset
- .to_dict(orient="records"): Converts to list of dictionaries format where each record becomes {column: value} pairs
- table: List of 10 dictionaries passed to the template for HTML table rendering

Line 13: Get List of Generated Plots

plots=os.listdir(os.path.join(base,"outputs","plots"))
Explanation:

- os.listdir(): Lists all files in the specified directory
- Directory path: {base_directory}/outputs/plots/
- plots: List of PNG filenames (grade.png, iq.png, scatter.png, fees.png, attendance.png, skills.png, heatmap.png)

Line 14: Render and Return HTML Template

return render_template("index.html", table=table, plots=plots)
Explanation:

- render_template(): Loads index.html from templates/ folder
- table=table: Passes top 10 student records to template
- plots=plots: Passes list of plot filenames to template
- Returns rendered HTML page with dynamic content injected

Lines 16-17: Plot Image Serving Route

@app.route("/plot/")
def plotimg(name):
Explanation:

- @app.route("/plot/"): Maps requests to /plot/{name} URL pattern
- name: URL parameter capturing the requested plot filename
- def plotimg(name):: Handler function receives filename from URL

Line 18: Serve Plot File

return send_from_directory(os.path.join(base,"outputs","plots"), name)
Explanation:

- send_from_directory(): Securely serves files from server disk (prevents directory traversal attacks)
- First argument: Safe directory path {base}/outputs/plots/
- Second argument: name filename parameter
- Returns PNG image file with appropriate headers for browser display

Line 20: Run Flask Application

app.run(debug=True)
Explanation:

- app.run(): Starts the Flask development server
- debug=True: Enables debug mode (auto-reload on code changes, detailed error messages)
- Server runs on default http://localhost:5000

File 2: generate_plots_and_models.py

Purpose: Data processing, visualization generation, and machine learning model training pipeline.

Description: This is a comprehensive script that reads the student dataset, performs data transformations, generates 6 statistical visualizations, trains 4 predictive ML models (CGPA prediction, pass/fail classification, defaulter detection, clustering), and saves outputs for use in the Flask app.

Complete Source Code (Line-by-Line)

Lines 1-12: Import All Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score
import pickle, os
```

Explanation:

- pandas: Data manipulation and analysis (CSV reading, column operations, grouping)
- numpy: Numerical computing (array operations, mathematical functions)
- matplotlib.pyplot: Creating and saving static visualization plots
- train_test_split: Splitting data into training (80%) and test (20%) sets
- RandomForestRegressor: Ensemble regression model for continuous targets (CGPA prediction)
- RandomForestClassifier: Ensemble classification model for binary/multiclass targets (pass/fail, defaulter)
- LinearRegression: Baseline regression model (imported but unused in final version)
- KMeans: Unsupervised clustering algorithm for student segmentation
- OneHotEncoder: Converts categorical variables (gender, grade, city) to numerical format
- StandardScaler: Normalizes features to zero mean and unit variance
- mean_squared_error, r2_score, accuracy_score: Model evaluation metrics
- pickle: Serializes trained models to .pkl files for later loading
- os: File and directory operations

Line 14: Define Base Directory

```
base=os.path.dirname(__file__)
```

Explanation: Same as app.py - stores the script's directory path for relative file references.

Line 16: Load Student Dataset

```
df=pd.read_csv(os.path.join(base,"student_dataset_100.csv"))
```

Explanation: Reads 100-student CSV file into DataFrame. Creates 13 columns including fees, grades, IQ, attendance, CGPA, skills, and demographic information.

Lines 18-20: Feature Engineering - Create Derived Columns

```
df["fees_outstanding"]=df["total_fees"]-df["fees_paid"]
df["pass_fail"]=(df["cgpa"]>=6).astype(int)
df["defaulter"]=(df["fees_paid"]<0.8*df["total_fees"]).astype(int)
```

Explanation:

- **Line 18:** fees_outstanding = Total fees minus paid fees (shows outstanding balance)

- **Line 19:** pass_fail = Binary target (1 if CGPA ≥ 6.0 = pass, 0 otherwise)
- **Line 20:** defaulter = Binary target (1 if student paid <80% of total fees, 0 otherwise)

Lines 22-23: Create Output Directory for Plots

```
out=f"{base}/outputs/plots"
os.makedirs(out,exist_ok=True)
```

Explanation:

- out: Full path to output directory {base}/outputs/plots/
- os.makedirs(): Creates directory if it doesn't exist
- exist_ok=True: Doesn't raise error if directory already exists

Line 25: Set Plot Resolution

```
plt.rcParams["figure.dpi"]=120
```

Explanation: Sets default DPI (dots per inch) to 120 for all subsequent plots. Higher DPI = better quality but larger file size.

Block 1: Grade Distribution Bar Chart (Lines 27-29)

Grade chart

```
plt.bar(df["grade"].value_counts().index, df["grade"].value_counts().values)
plt.title("Grade Distribution")
plt.savefig(f"{out}/grade.png"); plt.close()
```

Explanation:

- df["grade"].value_counts(): Counts occurrences of each grade (A, B, C, D)
- .index: Grade categories (x-axis)
- .values: Count for each grade (y-axis)
- plt.bar(): Creates horizontal bar chart
- plt.title(): Adds chart title
- plt.savefig(): Saves plot as PNG image
- plt.close(): Clears plot memory to prevent memory leaks

Block 2: IQ Distribution Histogram (Lines 31-32)

IQ chart

```
plt.hist(df["IQ"],bins=10); plt.title("IQ Histogram")
plt.savefig(f"{out}/iq.png"); plt.close()
```

Explanation:

- plt.hist(): Creates histogram of IQ scores
- bins=10: Divides range into 10 equal-width bins
- Shows distribution shape of student IQ across population

Block 3: CGPA vs IQ Scatter Plot (Lines 34-35)

Scatter

```
plt.scatter(df["IQ"],df["cgpa"]); plt.title("CGPA vs IQ")
plt.savefig(f"{out}/scatter.png"); plt.close()
```

Explanation:

- `plt.scatter()`: Creates scatter plot with IQ on x-axis, CGPA on y-axis
- Each point represents one student
- Visualizes correlation between intelligence and academic performance

Block 4: Top 20 Outstanding Fees Bar Chart (Lines 37-41)

Fees outstanding

```
top=df.sort_values("fees_outstanding",ascending=False).head(20)
plt.bar(top["student_id"],top["fees_outstanding"])
plt.xticks(rotation=45)
plt.title("Fees Outstanding Top 20")
plt.savefig(f"{out}/fees.png"); plt.close()
```

Explanation:

- `.sort_values()`: Sorts DataFrame by fees_outstanding in descending order
- `.head(20)`: Takes top 20 students with highest outstanding fees
- `plt.bar()`: Bar chart with student IDs and amounts
- `plt.xticks(rotation=45)`: Rotates x-axis labels 45° for readability

Block 5: Average Attendance by Grade (Lines 43-45)

Attendance

```
att=df.groupby("grade")["attendance"].mean()
plt.bar(att.index,att.values); plt.title("Attendance by Grade")
plt.savefig(f"{out}/attendance.png"); plt.close()
```

Explanation:

- `df.groupby("grade")`: Groups students by their grade (A, B, C, D)
- `["attendance"].mean()`: Calculates average attendance for each grade group
- `att.index`: Grade categories (x-axis)
- `att.values`: Average attendance percentage (y-axis)

Block 6: Skills Frequency Chart (Lines 47-50)

Skills

```
skills=df["skills"].str.split(", ").explode().value_counts().head(20)
plt.bar(skills.index,skills.values); plt.xticks(rotation=45)
plt.title("Skills Frequency")
plt.savefig(f"{out}/skills.png"); plt.close()
```

Explanation:

- df["skills"].str.split(", "): Splits comma-separated skills list
- .explode(): Creates one row per skill (expands 'Python, Java' into separate rows)
- .value_counts(): Counts occurrences of each skill
- .head(20): Top 20 most common skills
- Creates bar chart showing skill popularity

Block 7: Correlation Heatmap (Lines 52-59)

Heatmap

```
num=['age','IQ','total_fees','fees_paid','fees_outstanding','attendance','cgpa']
corr=df[num].corr()
plt.imshow(corr); plt.colorbar()
plt.xticks(range(len(num)),num,rotation=45)
plt.yticks(range(len(num)),num)
plt.title("Correlation Heatmap")
plt.savefig(f"{out}/heatmap.png"); plt.close()
```

Explanation:

- num: List of 7 numerical columns for correlation analysis
- df[num].corr(): Pearson correlation matrix (7×7)
- plt.imshow(): Renders correlation matrix as heatmap (color intensity = correlation strength)
- plt.colorbar(): Adds color scale legend
- Shows which variables are strongly related (e.g., CGPA vs attendance)

ML Section: Feature Preparation (Lines 61-67)

ML

```
cat=["gender","grade","city"]
num=["age","IQ","attendance","total_fees","fees_paid"]
ohe=OneHotEncoder(sparse_output=False, drop="first")
cat_enc=ohe.fit_transform(df[cat])
X=np.hstack([df[num].values,cat_enc])
```

Explanation:

- cat: Categorical features (need encoding)
- num: Numerical features (already numeric)
- OneHotEncoder(): Converts categories to binary columns
- drop="first": Prevents multicollinearity by dropping first category

- cat_enc: One-hot encoded categorical matrix
- X: Final feature matrix combining numerical and encoded categorical features
- np.hstack(): Horizontally stacks numerical and encoded matrices

Block 8: CGPA Prediction Model (Lines 69-79)

CGPA model

```
y=df["cgpa"].values
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=0.2,random_state=42)
model=RandomForestRegressor(n_estimators=200,random_state=42)
model.fit(Xtr,ytr)
pred=model.predict(Xte)
rmse=(mean_squared_error(yte,pred))**0.5
r2=r2_score(yte,pred)
pickle.dump(model,open(f"{base}/models/cgpa.pkl","wb"))
```

Explanation:

- y: Target variable (continuous CGPA scores 0-10)
- train_test_split(): 80% training, 20% testing, random_state=42 ensures reproducibility
- RandomForestRegressor(): Ensemble of 200 decision trees
- model.fit(): Train on training data
- pred: Predictions on test set
- rmse: Root Mean Square Error metric (lower = better)
- r2: R^2 score (0-1, higher = better fit)
- pickle.dump(): Saves trained model to disk for later use

Block 9: Pass/Fail Classification Model (Lines 81-86)

Pass Fail

```
y=df["pass_fail"].values
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=0.2,random_state=42)
clf=RandomForestClassifier(n_estimators=200,random_state=42)
clf.fit(Xtr,ytr)
pickle.dump(clf,open(f"{base}/models/pass.pkl","wb"))
```

Explanation:

- y: Binary target (1=pass if CGPA≥6, 0=fail)
- RandomForestClassifier(): Ensemble classifier with 200 trees
- Trained to predict whether student will pass or fail based on features
- Saves model to models/pass.pkl

Block 10: Defaulter Prediction Model (Lines 88-93)

Defaulter

```
y=df["defaulter"].values
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=0.2,random_state=42)
clf2=RandomForestClassifier(n_estimators=200,random_state=42)
clf2.fit(Xtr,ytr)
pickle.dump(clf2,open(f"{base}/models/def.pkl","wb"))
```

Explanation:

- y: Binary target (1=defaulter if paid <80% fees, 0=non-defaulter)
- Predicts financial risk - which students likely won't pay full fees
- Saves model to models/def.pkl

Block 11: K-Means Clustering (Lines 95-102)

Clustering

```
sc=StandardScaler()
Xc=sc.fit_transform(df[['IQ','attendance','cgpa','fees_outstanding']])
km=KMeans(n_clusters=3,random_state=42,n_init=10)
df['cluster']=km.fit_predict(Xc)
df.to_csv(f"{base}/student_dataset_100_with_labels.csv",index=False)
pickle.dump(km,open(f"{base}/models/kmeans.pkl","wb"))
pickle.dump(sc,open(f"{base}/models/scaler.pkl","wb"))
```

Explanation:

- StandardScaler(): Normalizes features (mean=0, std=1) required for K-Means
- Feature selection: 4 key variables (IQ, attendance, CGPA, fees_outstanding)
- KMeans(): Unsupervised clustering into 3 groups
- n_init=10: Runs algorithm 10 times, picks best result
- km.fit_predict(): Clusters students and returns cluster labels (0, 1, or 2)
- df['cluster']: Adds cluster assignments to DataFrame
- .to_csv(): Saves augmented dataset with cluster labels
- Saves both KMeans model and scaler for future data preprocessing

Line 104: Completion Message

```
print("DONE.")
```

Explanation: Prints completion message to console indicating successful execution of entire pipeline.

File 3: student_dataset_100.csv

Purpose: Student dataset containing 100 records with demographic, academic, and financial information.

Description: CSV file with 13 columns representing student attributes. Used as the data source for all visualizations and machine learning models.

Dataset Structure

Column Definitions

Column	Type	Description	Example
student_id	String	Unique identifier	S001
name	String	Student full name	Simran Verma
age	Integer	Student age in years	23
gender	Categorical	Male or Female	Male
grade	Categorical	Academic grade (A/B/C/D)	C
IQ	Integer	Intelligence quotient score	85
total_fees	Integer	Total tuition fees in currency	38085
attendance	Integer	Attendance percentage (0-100)	97
cgpa	Float	Cumulative GPA (0-10 scale)	8.22
city	Categorical	Student residence city	Patna
fees_paid	Integer	Amount of fees already paid	19374
skills	String (CSV)	Comma-separated skills list	Python, Excel, Java, ML
fees_outstanding	Derived	Total fees minus fees paid	18711

Sample Records (First 10 of 100)

S001 | Simran Verma | Age 23 | Male | Grade C | IQ 85 | CGPA 8.22 | Attendance 97% | Patna | Skills: Python, Excel, Java, ML
S002 | Simran Singh | Age 17 | Female | Grade A | IQ 88 | CGPA 5.43 | Attendance 99% | Mumbai | Skills: SQL, Leadership, ML
S003 | Priya Singh | Age 21 | Female | Grade D | IQ 138 | CGPA 7.92 | Attendance 71% | Patna | Skills: Deep Learning, ML, Leadership
S004 | Neha Jha | Age 22 | Female | Grade A | IQ 143 | CGPA 8.41 | Attendance 57% | Delhi | Skills: C++, Data Analysis, Python, SQL
S005 | Vikas Yadav | Age 17 | Female | Grade C | IQ 119 | CGPA 5.34 | Attendance 56% | Kolkata | Skills: ML, Java, C++
S006 | Priya Mishra | Age 24 | Female | Grade C | IQ 107 | CGPA 6.91 | Attendance 93% | Mumbai | Skills: SQL, Leadership, Python, C++
S007 | Simran Mishra | Age 21 | Female | Grade C | IQ 85 | CGPA 7.57 | Attendance 75% | Chennai | Skills: SQL, Communication, Leadership, ML
S008 | Rohan Yadav | Age 21 | Female | Grade A | IQ 140 | CGPA 6.08 | Attendance 68% | Patna | Skills: SQL, Leadership
S009 | Ritika Mishra | Age 24 | Male | Grade B | IQ 127 | CGPA 7.88 | Attendance 83% | Kolkata | Skills: Java, Python, Excel
S010 | Neha Kumar | Age 23 | Female | Grade D | IQ 127 | CGPA 9.91 | Attendance 81% | Bangalore | Skills: ML, Data Analysis, Communication

Data Statistics

- **Total Records:** 100 students
- **Age Range:** 17-24 years
- **IQ Range:** 80-149 (standard deviation ~20)
- **CGPA Range:** 5.04-9.98 (0-10 scale)
- **Attendance Range:** 50-99% (percentage)
- **Fee Range:** ₹30,040 - ₹79,837
- **Cities Represented:** Mumbai, Kolkata, Delhi, Chennai, Bangalore, Patna (6 major Indian cities)
- **Skills:** 7 common skills tracked (Python, Java, C++, SQL, Excel, ML, Deep Learning, Leadership, Data Analysis, Communication)

Key Statistics

Academic Performance (CGPA):

- Mean: ~7.6
- Median: ~7.7
- Min: 5.04
- Max: 9.98

Attendance:

- Mean: ~72%
- Range: 50-99%
- Indicator of engagement and course completion likelihood

Financial Status:

- Students with outstanding fees range from ₹199 to ₹46,074
- Average outstanding: ~₹16,000

Summary of All Three Files

Integration Flow

```
student_dataset_100.csv
↓
generate_plots_and_models.py
↓
├─→ Generates 6 visualizations (PNG files in outputs/plots/)
├─→ Trains 4 ML models (saved as .pkl files in models/)
└─→ Creates augmented dataset with cluster labels
↓
app.py
↓
Serves dashboard at http://localhost:5000
├─ Displays top 10 student records in table
└─ Renders 6 generated plots
```

Technologies Used

- **Language:** Python 3
- **Web Framework:** Flask
- **Data Processing:** Pandas, NumPy
- **Machine Learning:** Scikit-learn
- **Visualization:** Matplotlib
- **Serialization:** Pickle (model storage)

Model Architectures

Model	Type	Purpose	Hyperparameters
CGPA Predictor	Random Forest Regressor	Predict student CGPA	200 trees, random_state=42
Pass/Fail Classifier	Random Forest Classifier	Binary classification (pass/fail)	200 trees, random_state=42
Defaulter Detector	Random Forest Classifier	Financial risk prediction	200 trees, random_state=42
Student Clustering	K-Means	Unsupervised segmentation into 3 groups	3 clusters, 10 initializations

---

End of Source Code Compilation